amU Faculté des sciences Aix Marseille Université

Année universitaire 2024/2025

Site : ☐ Luminy ☒ St-Charles ☐ St-Jérôme ☐ Cht-Gombert ☐ Aix-Montperrin ☐ Aubagne-SATIS

Sujet de : ☐ 1er semestre ☐ 2ème semestre ☒ Session 2 Durée de l'épreuve : 2h

Examen de : L2 Nom du diplôme : Licence Informatique parcours Informatique

Code du module : SIN4U27 Libellé du module : Programmation C

Calculatrices autorisées : NON Documents autorisés : NON

Nom : _____

Prénom : _____

N° étudiant : _____

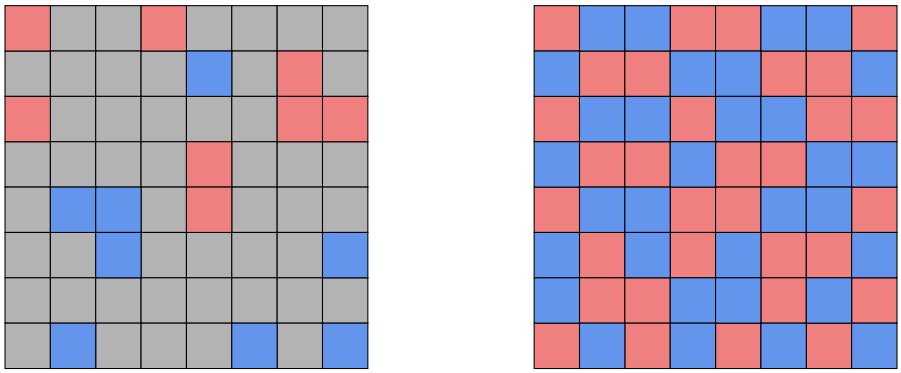
Répondre directement sur l'énoncé. Garder l'énoncé agrafé.

Le sujet prend pour prétexte *Unruly*, un casse-tête se présentant sous la forme d'une grille carré devant être coloriée en rouge et bleu. Au début du jeu, seule une partie des cases sont coloriées. L'objectif est de colorier toutes les autres cases, en respectant deux règles :

— Il n'y a pas trois cases consécutives d'une ligne ou d'une colonne de même couleurs ;

— chaque ligne et chaque colonne possède autant de cases bleues que de cases rouges.

Voici un exemple de position initiale avec sa solution à droite.



1 Représentation du plateau

Pour représenter le plateau de jeu, nous définissons deux types : celui des cellules et celui du plateau. Ce dernier contient la dimension d, qui est la longueur des côtés du plateau carré, et un tableau unidimensionnel de cellules `grid`, de longueur $d \times d$. La cellule de coordonnée (c, r) (c pour *column*/colonne, r pour *row*/rangée) sera représenté à l'index $c + d \times r$ du tableau `grid`. Nous supposerons pour tout l'énoncé que la dimension est paire.

```c
#define CELL_KIND_COUNT 3

typedef enum {
  CELL_EMPTY, CELL_RED, CELL_BLUE,
} Cell;

typedef struct {
  int dimension;
  Cell * grid;
} Board;
```

Exercice 1.1. *Afin de s'assurer que nos fonctions sont utilisées avec des données correctes, nous voulons disposer d'une fonction décidant si une paire de coordonnées (r, c) correspond bien à une case du plateau. Compléter pour cela la fonction* ***`Board_isValidCoordinate`****.*

```
bool Board_isValidCoordinate(Board * board, int row, int column) {
  return 
      ;
}
```

Exercice 1.2. *Nous aurons régulièrement besoin d'accéder à la cellule d'un tableau à partir de ces coordonnées (c, r). Écrire une fonction renvoyant l'indice de la cellule dans le tableau* ***`grid`*** *du plateau* ***`board`****.*

```
int index_of_coordinate(Board * board, int row, int column) {
  return ;
}
```

Exercice 1.3. *En déduire un accesseur et un mutateur pour les cases du plateau, qui s'assurent que les coordonnées fournies sont légales.*

```
Cell Board_getCell(Board * board, int row, int column) {
  assert();
  return ;
}

void Board_setCell(Board * board, int row, int column, Cell newValue) {
  assert();
  ;
}
```

2 Création et destruction des plateaux

Exercice 2.1. *Compléter la fonction d'initialisation du plateau. Elle alloue la grille selon la dimension fournie, et la remplit de cellules vides.*

```
void Board_init(Board * board, int dimension) {
  = dimension;
  = ;
  assert();
  for (int row = 0; row < board->dimension; row++) {
    for (int column = 0; column < board->dimension; column++) {
      ;
    }
  }
}
```

Exercice 2.2. *En déduire la fonction de création d'un nouveau plateau, qui retourne l'adresse d'un plateau alloué dynamiquement. Le plateau doit être de la dimension demandée et vide.*

```
Board * Board_make(int dimension) {
  Board * board = ;
  assert( );
  ;
  return board;
}
```

Exercice 2.3. *Compléter la fonction* ***`Board_clean`****, qui libère la mémoire utilisée par la grille uniquement, et la fonction* ***`Board_destroy`****, qui libère la mémoire de la grille et de* ***`board`*** *lui-même.*

```
void Board_clean(Board * board) {
  ;
}

void Board_destroy(Board * board) {
  ;
  ;
}
```

3 Lecture et écriture des plateaux

Dans cette section, nous souhaitons convertir les plateaux en chaînes de caractères et inversement. Par exemple, les deux plateaux dessinés ci-dessus devront être représentables avec les deux chaînes suivantes (les retours à la ligne ont été ajoutés pour mieux illustrer les deux chaînes, qui contiennent chacune 72 caractères).

```
"O__O____\n            "OXXOOXXO\n
 ____X_O_\n             XOOXXOOX\n
 O_____OO\n             OXXOXXOO\n
 ____O___\n             XOOXOOXX\n
 _XX_O___\n             OXXOOXXO\n
 __X____X\n             XOXOXOOX\n
 ________\n             XOOXXOXO\n
 _X___X_X\n"            OXOXOXOX\n"
```

Nous définissons le tableau contenant les symboles que nous utiliserons, et une fonction de conversion.

```
char const * CELL_SYMBOLS = " OX";

Cell cell_of_symbol(char c) {
  for (size_t i = 0; i < strlen(CELL_SYMBOLS); i++) {
    if (c == CELL_SYMBOLS[i]) return i;
  }
  return -1;
}
```

Exercice 3.1. *Compléter la fonction* ***Board_print****, qui affiche le plateau caractère par caractère, dans le fichier* ***file****.*

```
void Board_print(Board * board, FILE * file) {
  for (int row = 0; row < board->dimension; row++) {
    for (int column = 0; column < board->dimension; column++) {
      char cellSymbol =                                          ;
      fprintf(file,                                              );
    }
    fprintf(file,                                                );
  }
}
```

Exercice 3.2. *Compléter la fonction* ***Board_readFromString****, qui construit un plateau à partir de sa représentation textuelle. Pour obtenir la dimension du plateau, il suffira de chercher la position du premier caractère* *'\n'* *dans le texte. On supposera que* ***string*** *représente bien un tableau valide.*

```
Board * Board_readFromString(char * string) {
  int dimension = strchr(                ,                ) -                ;
  Board * board = Board_make(dimension);
  for (int row = 0; row < board->dimension; row++) {
    for (int column = 0; column < board->dimension; column++) {
      char symbol =                                              ;
      Board_setCell(board, row, column,                          );
    }
  }
  return board;
}
```

4 Analyse du plateau

Exercice 4.1. *Les fonctions suivantes utiliseront le type* *bool* *et ses valeurs. Quelle directive faut-il alors ajouter au début du fichier ?*

```
#include
```

Exercice 4.2. *Compléter la fonction* ***Board_rowCount****, qui compte le nombre de cellules égales à* ***kind*** *dans la rangée* ***row****. Cette fonction permettra de compter le nombre de cases bleus, rouges ou vides de n'importe quelle rangée.*

```
int Board_rowCount(Board * board, int row, Cell kind) {
  int count =                                                    ;
  for (int column = 0; column < board->dimension; column++) {
    if (                                                         ) {
      count =                                                    ;
```

```
    }
  }
  return ;
}
```

Exercice 4.3. *Compléter la fonction suivante, qui teste si une rangée* `row` *possède trois cellules consécutives de même couleur (rouge ou bleu). Pour cela, nous essayons la cellule du milieu et vérifions que ses deux voisines sont de la même couleur.*

```
bool Board_rowHasEqualTriple(Board * board, int row) {
  for ( ; ; ) {
    Cell current = Board_getCell(board, row, column);
    if (current == CELL_EMPTY) continue;
    if (
        && ) {
      return ;
    }
  }
  return ;
}
```

Exercice 4.4. *En déduire une fonction testant si les cellules d'une même ligne vérifient bien les deux règles du jeu : au plus la moitié des cellules de chaque couleur, et pas trois cellules consécutives de même couleur.*

```
  bool Board_rowIsValid(Board * board, int row) {
    assert(0 <= row && row < board->dimension);
    return
      
      
      ;
  }
}
```

Un travail symétrique nous permet d'écrire la fonction `Board_columnIsValid`, que nous ne détaillons pas.

Exercice 4.5. *Écrire une fonction qui permet de vérifier si colorier une cellule donnée est permis par les règles. Pour cela, nous affectons la couleur dans la cellule, vérifions que sa ligne et sa colonne respectent les règles, puis nous remettons la cellule dans son état initial.*

```
bool Board_isSetLegal(Board * board, int row, int column, Cell newContent) {
  Cell savedContent = ;
  Board_setCell(board, row, column, );
  bool isLegal =
    
    ;
```

```
  Board_setCell(board, row, column, [          ]);
  return isLegal;
}
```

Exercice 4.6. *Écrire une fonction qui, prenant les coordonnées d'une cellule, vérifie si elle est vide et si une seule couleur peut lui être attribuée.*

```
bool Board_isCellForced(Board * board, int row, int column) {
  return [          ] == CELL_EMPTY
    && [          ]
      != [          ];
}
```

Exercice 4.7. *Écrire une fonction dont le rôle est de trouver une cellule vide sur le plateau. Elle retourne* `true` *si le plateau contient une cellule vide. Dans ce cas, ses paramêtres* `* row` *et* `* column` *doivent contenir les coordonnées d'une cellule vide.*

```
bool Board_findEmptyCell(Board * board, int * row, int * column) {
  for ([          ];[          ];[          ]) {
    for ([          ];[          ];[          ]) {
      if ([          ]) {
        return true;
      }
    }
  }
  return false;
}
```

Exercice 4.8. *En déduire une fonction déterminant si le plateau est rempli : elle renvoie* `vrai` *si aucune cellule n'est vide.*

```
bool Board_isComplete(Board * board) {
  [          ];
  return [          ];
}
```

Exercice 4.9. *Compléter la fonction suivante, qui crée une copie du plateau fournie.*

```
Board * Board_clone(Board * board) {
  Board * copy = [          ];
  memcpy([          ]);
  return copy;
}
```